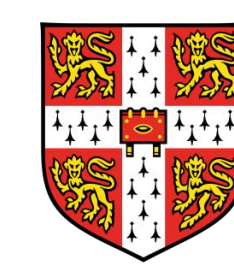
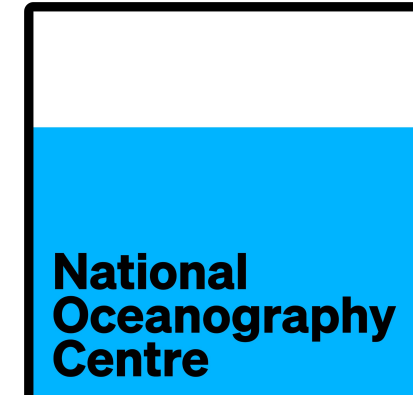


Scaling up CATS, The *Climate-Aware Task Scheduler*, for cluster computing and international reach



UNIVERSITY OF CAMBRIDGE



Sadie L. Bartholomew^{1,2}, Lincoln Colling³, Abhishek Dasgupta⁴, Anthony J. Greenberg⁵, Loïc Lannelongue⁶, Colin Sauzé⁷, Andrew M. Walker⁸

¹ National Centre for Atmospheric Science, UK; ² Department of Meteorology, University of Reading, UK; ³ Nomio, UK; ⁴ Oxford Research Software Engineering Group, University of Oxford, UK; ⁵ Bayesic Research, USA; ⁶ University of Cambridge, UK; ⁷ National Oceanography Centre, UK; ⁸ Department of Earth Sciences, University of Oxford, UK

Border image: 'Climate Stripes' infographic by Prof. Ed Hawkins (University of Reading), see showyourstripes.info

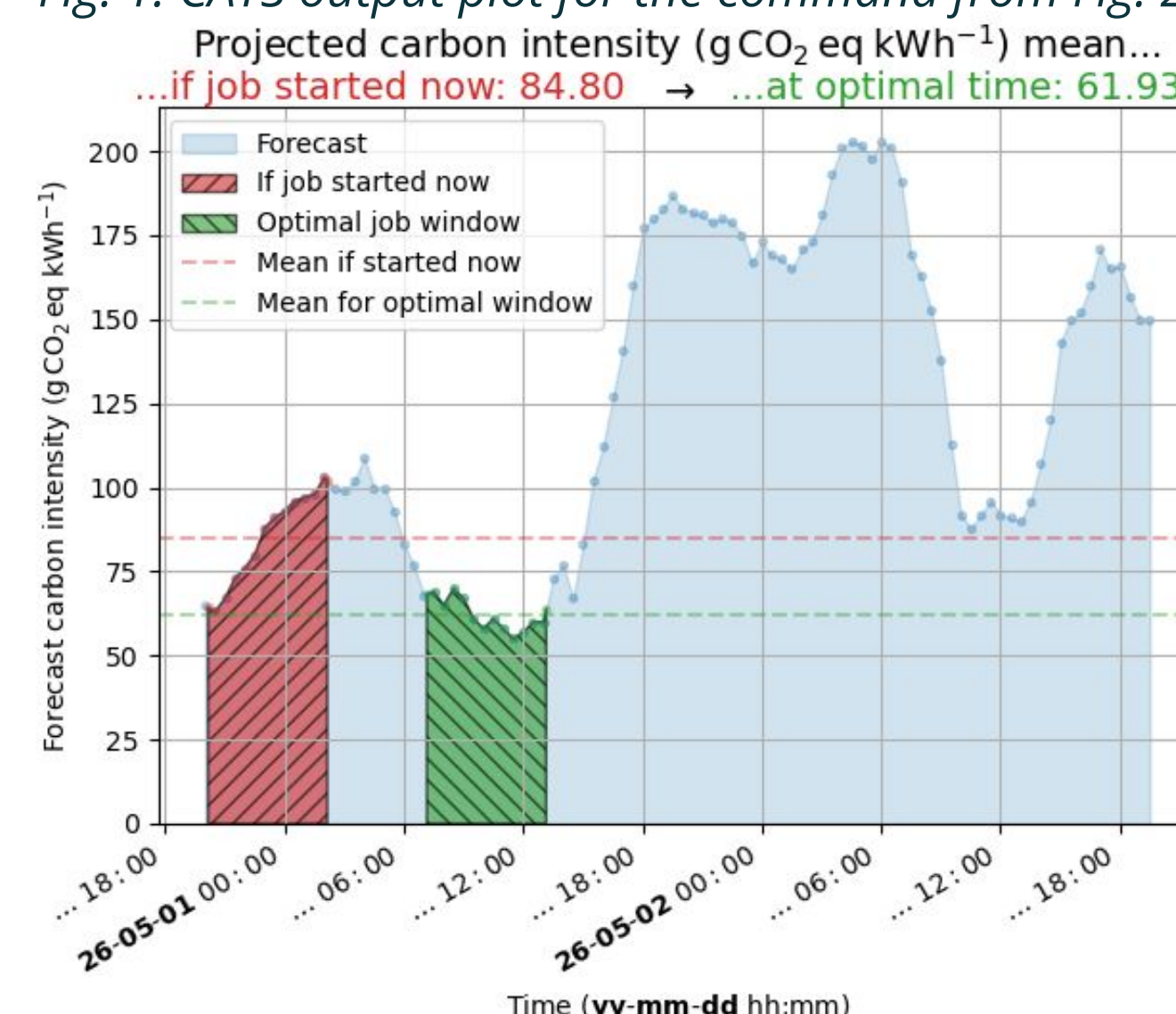
Context: the problem

Indirect greenhouse gas (GHG) emissions that arise from the consumption of grid electricity (*Scope 2 emissions*) are estimated to be responsible for over a third of global GHG emissions¹. Operational energy use in computing for research, especially at scale or in high-performance (HPC) or high-throughput (HTC) application, produces substantial Scope 2 emissions, causing significant environmental harm.

One way to mitigate this is to do more compute when available electricity is greener, having higher contribution from clean energy (renewables) relative to dirty energy (fossil fuels), quantified by *carbon intensity* with units of gCO₂e/kWh. Electricity grid carbon intensity varies considerably both in time and geographically according to demand and weather dependence etc., with ≤ 50 gCO₂e/kWh deemed low(-carbon) and ≥ 600 high. Shifts in time or location, or anticipating and adapting to demand, can reduce carbon intensity and thus lower CO₂ emissions for set energy consumption.

How CATS works: minimising carbon intensity

Fig. 1: CATS output plot for the command from Fig. 2



CATS (the Climate-Aware Task Scheduler)^{2,3,4} is a Python library and command-line tool that calculates the optimal time to run a compute job within the next two-day (or configurably smaller) window to minimise its carbon intensity across its runtime, then optionally schedule the job to run at that datetime. By delaying the job until the later optimal window, less CO₂ emissions are produced per kilowatt hour of electricity consumed by the job.

CATS is not the only tool to implement *carbon-aware* scheduling¹, but it offers a lightweight general solution, applicable to any job that can be initiated by command, whether run locally or (soon in v.2[†]) on a cluster. It is designed to be conceptually simple and user friendly, so can serve as an educational tool.

Installation and basic usage of CATS

To install CATS, run `$ pip install climate-aware-task-scheduler` to get it from PyPI or download the source and `pip install` from the GitHub repo². For basic local usage, provide the anticipated job duration (in minutes) and location as a (UK*) postcode to receive the optimal start time estimate via the command `$ cats --duration <duration> --location <postcode>`. Adding `--plot` will also return a plot of the relevant carbon intensity time series with optimal window indicated (for illustration see Fig. 1). For example:

```
$ cats --duration 360 --location "SW1A" --plot
```

The output shows a plot of carbon intensity over time, with the optimal job window highlighted in green. Below the plot, the following information is displayed:

Best job start time	= 2026-05-01 08:09:11
Carbon intensity if job started now	= 84.80 gCO ₂ e/kWh
Carbon intensity at optimal time	= 61.93 gCO ₂ e/kWh

Fig. 2: example command and output using the CATS command-line interface (CLI) for a job run in central London at postcode 'SW1A' and expected to take 6 hours to complete.

As well as finding the optimal time, one can directly schedule jobs at the corresponding datetime using the `--scheduler` argument. We currently support `at` and `sbatch`. For example, to run a Python script `work.py` expected to take an hour to run locally on a computer at the University of Warwick (postcode 'CV4'), the command to run is:

```
$ cats -d 60 --loc CV4 --scheduler sbatch --command 'python work.py'.
```

Use on clusters and limitations

[†]Version 1 of CATS targeted smaller-scale tasks on local machines but the most energy-intensive workloads are typically run on some tier of supercomputer facility, so for CATS to be most useful at mitigating emissions, it is necessary to support use on clusters. Version 2+ of CATS will target cluster support through integration with *batch scheduling and queuing systems*.

Work for v2 has focused on the Slurm Workload Manager and so far an ephemeral (temporary, on-demand compute) Slurm cluster has been developed for testing. The intention is to work with system administrators at volunteer centres to develop a 'green' queue incorporating CATS v2 into HPC systems to provide users with the option to intelligently time shift their jobs to reduce carbon emissions.

CATS has limitations particularly for cluster application. Firstly, some HPC systems such as the UK's National Tier-1 ARCHER2 are already powered by 100% renewable energy, therefore the time shifting approach is not applicable. Moreover, CATS only works when a system isn't running at (or near) 100% load so it is best suited to clusters with spare capacity. Job scheduling constraints or workflow dependencies can further limit the extent to which workloads can be deferred, while uncertainty in carbon intensity forecasts may reduce optimisation accuracy. CATS is dependent on an estimated job duration, which often cannot be accurately determined at least without a dry run. Finally, only UK systems are presently supported, though the aim is to extend support via identification of suitable APIs for outside the UK^{*}.

UK → International reach

*Ideally CATS could be used anywhere but it is constrained by the availability of appropriate APIs for electricity system grid forecasts and at present only the UK National Grid ESO API⁵ is supported, so use outside of the UK is not possible.

To achieve international support, we are seeking candidates for APIs for electricity grids of other countries or areas that CATS could hook up to. If you know of any API with potential for this, please let us know via the Issue (#22) on the CATS Issue Tracker using the QR code to the right.



References (further reading)

To cite CATS please use ref. 2. To get started with CATS use ref. 3 or 4.

1. A. Mencaroni et al. (2025), *Towards net-zero manufacturing: Carbon-aware scheduling for GHG emissions reduction*, Journal of Cleaner Production, 529, <https://doi.org/10.1016/j.jclepro.2025.146787>
2. S. L. Bartholomew et al. (2025), *CATS: The Climate Aware Task Scheduler*. Journal of Open Source Software, 10(111), 8251, <https://doi.org/10.21105/joss.08251> (QR code to right)
3. CATS repository including codebase and issue tracker: <https://github.com/GreenScheduler/cats>
4. CATS documentation: <https://cats.readthedocs.io>
5. National Energy System Operator (NESO), Official Carbon Intensity API for Great Britain, api.carbonintensity.org.uk (see also carbonintensity.org.uk for further information)

